California State University, Sacramento
College of Engineering and Computer Science

Computer Science 35: Introduction to Computer Architecture

Spring 2026 – Lab 5: Farkle

Overview

In this lab you will implement a simplified version of the dice game Farkle in x86-64 assembly language using Intel syntax and the CSc 35 library. Farkle is a push-your-luck game where a player rolls six dice and decides whether to bank their points or risk rolling again. If a roll produces no scoring dice, the player "Farkles" and loses all points accumulated during that turn.

Your program will simulate a two-player game. Players take alternating turns, each accumulating their own running total. The first player to reach 10,000 points wins immediately.

How a Turn Works

1. The player rolls all six dice.

2. The program scores the roll and displays what scored and why. If no dice score, the player Farkles and loses all points for the turn.

3. If the roll scored, the player chooses to bank the turn score (ending the turn) or roll again with all six dice.

4. After a turn ends, the game checks if the current player has reached 10,000 points. If so, that player wins. Otherwise, play passes to the other player.

Farkle Scoring Rules

The following table shows the scoring combinations your program must recognize. This lab uses a simplified scoring system that checks for three-of-a-kind and single 1s and 5s only.

Combination	Points
Single 1	100
Single 5	50
Three 1s	1,000
Three 2s	200
Three 3s	300
Three 4s	400
Three 5s	500
Three 6s	600

Scoring priority: Three-of-a-kind takes priority. For example, if a player rolls three 1s, they score 1,000 (not 3 × 100). If they roll four 1s, they score 1,000 + 100 = 1,100 (three-of-a-kind plus one extra single). Similarly, three 5s scores 500, and a fourth 5 adds 50 more.

CSc 35 Library Reference

Function	Description
`PrintStrZ`	Prints a null-terminated string. Load address into rcx with lea before calling.
`PrintInt64`	Prints the signed 64-bit integer in rcx.
`ScanInt64`	Reads an integer from the user. Result is returned in rcx.
`GetRandom`	Returns a random integer in rcx in the range [0, rcx). Load the exclusive upper bound into rcx before calling.
`ProgramEnd`	Terminates the program. Must be the last call.

Note: Library calls may modify rcx. If you have an important value in rcx, save it to another register or to a memory variable before making a library call.

Program Pseudocode

The following pseudocode describes the complete program logic. Translate each section into x86-64 assembly using the CSc 35 library calls and Intel syntax.

```
// Variable Declarations
Declare Integer die1, die2, die3, die4, die5, die6
Declare Integer count1, count2, count3, count4, count5, count6
Declare Integer rollScore, turnScore, choice
Declare Integer scoreP1, scoreP2, currentPlayer

// ============================================================
// Main Program
// ============================================================
Set scoreP1 = 0
Set scoreP2 = 0
Set currentPlayer = 1

Do
    // ── Display Scores ──────────────────────────────
    Display "Player 1: ", scoreP1, "  |  Player 2: ", scoreP2
    Display "--- Player ", currentPlayer, "'s Turn ---"
    Set turnScore = 0

    // ── Turn Loop ───────────────────────────────────────
    Do
        // ── Roll 6 Dice ─────────────────────────────
        Set die1 = random(1, 6)
        Set die2 = random(1, 6)
        Set die3 = random(1, 6)
        Set die4 = random(1, 6)
        Set die5 = random(1, 6)
        Set die6 = random(1, 6)

        // ── Display the Roll ────────────────────────
        Display "You rolled: ", die1, " ", die2, " ",
               die3, " ", die4, " ", die5, " ", die6

        // ── Count Face Values ───────────────────────
        Set count1 = 0
        Set count2 = 0
        Set count3 = 0
        Set count4 = 0
        Set count5 = 0
        Set count6 = 0
```

```
// Count die1
If die1 == 1 Then
    Set count1 = count1 + 1
Else If die1 == 2 Then
    Set count2 = count2 + 1
Else If die1 == 3 Then
    Set count3 = count3 + 1
Else If die1 == 4 Then
    Set count4 = count4 + 1
Else If die1 == 5 Then
    Set count5 = count5 + 1
Else If die1 == 6 Then
    Set count6 = count6 + 1
End If

// Repeat the same If/Else If for die2 through die6

// ── Calculate Roll Score ──────────────────────
Set rollScore = 0
Display "Scoring:"

// Check three-of-a-kind (must check before singles)
If count1 >= 3 Then
    Set rollScore = rollScore + 1000
    Set count1 = count1 - 3
    Display "  Three 1s = 1000"
End If
If count2 >= 3 Then
    Set rollScore = rollScore + 200
    Set count2 = count2 - 3
    Display "  Three 2s = 200"
End If
If count3 >= 3 Then
    Set rollScore = rollScore + 300
    Set count3 = count3 - 3
    Display "  Three 3s = 300"
End If
If count4 >= 3 Then
    Set rollScore = rollScore + 400
    Set count4 = count4 - 3
    Display "  Three 4s = 400"
End If
If count5 >= 3 Then
    Set rollScore = rollScore + 500
    Set count5 = count5 - 3
    Display "  Three 5s = 500"
End If
If count6 >= 3 Then
    Set rollScore = rollScore + 600
    Set count6 = count6 - 3
    Display "  Three 6s = 600"
End If

// Add remaining single 1s (100 points each)
If count1 > 0 Then
    Set rollScore = rollScore + (count1 * 100)
    Display "  ", count1, " single 1(s) = ", count1 * 100
End If

// Add remaining single 5s (50 points each)
If count5 > 0 Then
    Set rollScore = rollScore + (count5 * 50)
    Display "  ", count5, " single 5(s) = ", count5 * 50
End If
```

```
        // —— Check for Farkle ————————————————————————
        If rollScore == 0 Then
            Display "  No scoring dice!"
            Display "FARKLE! You lose all points this turn."
            Set turnScore = 0
            Set choice = 1
        Else
            Display "Roll score: ", rollScore
            Set turnScore = turnScore + rollScore
            Display "Turn score so far: ", turnScore

            Display "Bank (1) or Roll Again (2)? "
            Input choice
        End If

    While choice != 1

    // —— End of Turn ————————————————————————————————————
    If turnScore > 0 Then
        Display "Player ", currentPlayer, " banked ",
               turnScore, " points!"
    End If

    // —— Add turn score to current player's total ———————
    If currentPlayer == 1 Then
        Set scoreP1 = scoreP1 + turnScore
    Else
        Set scoreP2 = scoreP2 + turnScore
    End If

    // —— Check for win ————————————————————————————————
    If currentPlayer == 1 AND scoreP1 >= 10000 Then
        Display "Player 1 wins with ", scoreP1, " points!"
        Stop
    End If
    If currentPlayer == 2 AND scoreP2 >= 10000 Then
        Display "Player 2 wins with ", scoreP2, " points!"
        Stop
    End If

    // —— Switch players ————————————————————————————————
    If currentPlayer == 1 Then
        Set currentPlayer = 2
    Else
        Set currentPlayer = 1
    End If

While True
```

Sample Output

The following shows what a portion of a game session might look like:

```
Welcome to Farkle: A Push-Your-Luck Game

Player 1: 0  |  Player 2: 0

--- Player 1's Turn ---

You rolled: 3 1 5 2 1 4
Scoring:
  2 single 1(s) = 200
  1 single 5(s) = 50
Roll score: 250
Turn score so far: 250
Bank (1) or Roll Again (2)? 2

You rolled: 1 1 1 5 3 2
Scoring:
  Three 1s = 1000
  1 single 5(s) = 50
Roll score: 1050
Turn score so far: 1300
Bank (1) or Roll Again (2)? 1
Player 1 banked 1300 points!

Player 1: 1300  |  Player 2: 0

--- Player 2's Turn ---

You rolled: 2 3 4 6 6 2
Scoring:
  No scoring dice!
FARKLE! You lose all points this turn.

Player 1: 1300  |  Player 2: 0

--- Player 1's Turn ---

You rolled: 5 5 5 1 3 4
Scoring:
  Three 5s = 500
  1 single 1(s) = 100
Roll score: 600
Turn score so far: 600
Bank (1) or Roll Again (2)? 1
Player 1 banked 600 points!

...

Player 1 wins with 10350 points!
```

Starting Template

Use the following template as your starting point. The .data section provides the string constants and memory variables you will need. Write your code in the .text section beginning at the Program label.

```
.intel_syntax noprefix

.data
# String constants
RollMsg:      .asciz "You rolled: "
Space:        .asciz " "
NewLine:      .asciz "\n"
ScoringHdr:   .asciz "Scoring:"
ScoreMsg:     .asciz "Roll score: "
TurnMsg:      .asciz "Turn score so far: "
BankMsg:      .asciz " banked "
PointsMsg:    .asciz " points!"
FarkleMsg:    .asciz "FARKLE! You lose all points this turn."
NoScoreMsg:   .asciz "  No scoring dice!"
ChoiceMsg:    .asciz "Bank (1) or Roll Again (2)? "

# Player strings
P1Label:      .asciz "Player 1"
P2Label:      .asciz "Player 2"
Separator:    .asciz "  |  "
ColonSpace:   .asciz ": "
TurnPrefix:   .asciz "--- "
TurnSuffix:   .asciz "'s Turn ---"
WinsMsg:      .asciz " wins with "
WinEnd:       .asciz " points!"

# Scoring explanation strings
Triple1Msg:   .asciz "  Three 1s = 1000"
Triple2Msg:   .asciz "  Three 2s = 200"
Triple3Msg:   .asciz "  Three 3s = 300"
Triple4Msg:   .asciz "  Three 4s = 400"
Triple5Msg:   .asciz "  Three 5s = 500"
Triple6Msg:   .asciz "  Three 6s = 600"
Single1Pre:   .asciz "  "
Single1Mid:   .asciz " single 1(s) = "
Single5Mid:   .asciz " single 5(s) = "

# Dice values
die1:          .quad 0
die2:          .quad 0
die3:          .quad 0
die4:          .quad 0
die5:          .quad 0
die6:          .quad 0

# Face counts
count1:        .quad 0
count2:        .quad 0
count3:        .quad 0
count4:        .quad 0
count5:        .quad 0
count6:        .quad 0

# Scores and game state
roll_score:    .quad 0
turn_score:    .quad 0
score_p1:      .quad 0
```

```
score_p2:      .quad 0
current_player:.quad 0

.text
.global Program
Program:
    # Your code starts here

    call ProgramEnd
```

Accessing memory variables: Use mov to read from and write to these labels. For example:

```
mov rcx, 6
call GetRandom
add rcx, 1
mov die1, rcx          # store die value into memory

mov rcx, die1          # load die value from memory
call PrintInt64        # print it
```

UNIX Commands

Command	Description
`nano lab5.s`	Open/create the source file in the nano editor
`as -o lab5.o lab5.s`	Assemble the source file into an object file
`Ld -o a.out lab5.o csc35.o`	Link object files into an executable
`./a.out`	Run the executable

Tips and Common Pitfalls

1. Save registers before library calls. PrintInt64, PrintStrZ, and GetRandom all use rcx. Save important values to other registers or memory variables before calling these functions.

2. Use comments liberally. Label each section of your code with comments explaining what it does. This will help you debug and will earn style points.

3. Test incrementally. Start by getting the dice rolling and display working. Then add scoring with the display messages. Then add the turn loop with bank/roll/Farkle. Then add the two-player game loop. Do not try to write the entire program at once.

4. Counting die faces. To count how many of each face were rolled, compare each die value to 1, 2, 3, 4, 5, and 6, incrementing the appropriate counter for each match. This requires many cmp/je instructions but is straightforward.

5. Multiplying without imul. To compute count1 × 100, you can use repeated addition or the approach: add reg, reg (doubles it), then use a combination of add instructions. Alternatively, you may use the imul instruction if it has been covered in class.

6. Jump on the opposite condition. Remember that in assembly, you typically jump over a block when the condition is false. For "if count1 >= 3", use cmp and jl (jump if less) to skip the three-of-a-kind block.

Grading Rubric

Criteria	Points	Score
Rolling and displaying six dice	2	
Scoring logic (three-of-a-kind, single 1s and 5s)	4	
Scoring display (explains what scored and why)	2	
Turn logic (re-roll, bank, or Farkle)	2	
Two-player support (alternating turns, separate scores)	2	
Game loop (first to 10,000 wins)	2	
Code style: comments, labels, organization	1	
Total	**15**	

Submission

This activity may only be submitted using the assembly format used in class: (1) x64, (2) Intel-syntax, and (3) the CSC 35 library.
Using AT&T format will result in a zero. <u>Any work from a prior semester will receive a zero.</u>

Select the ".asm" file in the MobaXTerm File Explorer pane and download it to the ECS Lab Computer. Once downloaded, open the Canvas Assignment and submit the ".asm" file.

Useful UNIX Commands

Editing

Action	Command	Notes
Edit File	`nano filename`	"Nano" is an easy to use text editor.
Assemble File	`as -o object source`	Don't mix up the object and asm fields. It will destroy your program!
Link File	`ld -o exe object(s)`	Link and create an executable file from one (or more) object files.

Folder Navigation

Action	Command	Notes
Change current folder	`cd foldername`	"Changes Directory"
Go to parent folder	`cd ..`	Think of it as the "back button".
Show current folder	`pwd`	Gives the current file path.
List files	`ls`	Lists the files in current directory.

File Organization

Action	Command	Notes
Create folder	**`mkdir foldername`**	Folders are called directories in UNIX.
Copy file	**`cp oldfile newfile`**	Make a copy of an existing file.
Move file	**`mv filename foldername`**	Moves a file to a destination folder.
Rename file	**`mv oldname newname`**	Note: same command as “move”.
Delete file	**`rm filename`**	Remove (delete) a file. There is no undo.